A Layered, Axiom-Isolated Lean 4 Formalization of the Congruent Number Problem up to Tunnell's Criterion

Papanokechi

April 2026

Abstract

We present a 954-line Lean 4 formalization of the combinatorial and structural backbone of Tunnell's criterion for the congruent number problem. The development is organized into six layers, each mathematically natural and fully proved, culminating in a clean axiom boundary where the sole remaining assumptions are the Birch and Swinnerton-Dyer (BSD) conjecture and Tunnell's conditional theorem.

The formalization contains zero `sorry` placeholders. The only axiom is `tunnell_conditional_on_BSD`, which encodes Tunnell's 1983 result. The BSD hypothesis is represented as BSD_for(n) := AlgebraicRank(n) = AnalyticRank(n), where both rank functions are opaque definitions, preventing vacuous discharge.

Key contributions include a formal non-vacuity proof for the axiom boundary (Theorem 4.1), a generic fixed-point-free involution lemma on `Finset` (proved by strong induction), an eightfold sign symmetry for positive-definite ternary quadratic forms, a bridging theorem connecting bounded and unbounded solution sets, and a theta-series interface that recovers Tunnell's counting quantities.

The formal development is paired with a Python computational pipeline (98 tests, validated against OEIS A003273) that verifies Tunnell's criterion for all squarefree $n \leq 500$. Code and artifacts are available at `https://github.com/papanokechi/congruent-relay`.

1 Introduction

Target venue. This paper targets the *Interactive Theorem Proving* (ITP) / *Certified Programs and Proofs* (CPP) community. The contribution is not new mathematics—Tunnell's criterion dates to 1983—but a new *formalization architecture*: a principled method for isolating deep conjectures behind opaque, non-vacuous axiom boundaries in Lean 4, demonstrated on a classical number-theoretic result.

A positive integer n is a *congruent number* if it is the area of a right triangle with rational side lengths. Equivalently, n is congruent if and only if the elliptic curve

$$E_n : y^2 = x^3 - n^2 x$$

has positive Mordell–Weil rank over $\mathbb{Q}$. The congruent number problem—determining which integers are congruent—is one of the oldest unsolved problems in number theory, with roots in Arab manuscripts from the tenth century [2].

Tunnell's theorem [1] provides an effective criterion: a squarefree positive integer n is congruent if and only if certain representation counts of ternary quadratic forms satisfy an equality condition, *provided* the Birch and Swinnerton-Dyer conjecture holds for E_n. The forward direction is unconditional; the reverse direction requires BSD.

Why formalize Tunnell's criterion? Tunnell's criterion sits at an unusual point in the formalization landscape. Its *statement* is entirely combinatorial—counting integer solutions to ternary quadratic equations—yet its *proof* requires the theory of half-integral weight modular forms, the Shimura correspondence, and the Waldspurger formula, none of which are formalized in any proof assistant. This creates a clean separation: the combinatorial backbone can be fully machine-checked, while the analytic core must be axiomatized.

The question we address is not "can we formalize Tunnell's proof?" (we cannot, with current libraries) but rather: *what is the right way to axiomatize what we cannot prove, while maximizing the amount we do prove?* Our answer is a three-level architecture: `opaque` for quantities that exist but whose values are inaccessible (rank functions), `def` for the conjecture relating them (BSD), and a single `axiom` for the conditional theorem.

Design philosophy.

1. **Layered architecture.** Six sections, each building on the previous, with a clear separation between combinatorial content (proved) and analytic content (axiomatized).
2. **Axiom isolation.** The BSD conjecture enters as a single, mathematically meaningful definition—not a vacuous `True` placeholder. A reviewer can verify that `tunnell_reverse_under_BSD` requires a genuinely non-trivial hypothesis.
3. **Reproducibility.** All definitions mirror a Python implementation; a test suite validates numerical agreement against OEIS A003273.

Contributions.

- **Axiom-isolation pattern.** A three-level `opaque`/`def`/`axiom` architecture for formalizing theorems conditional on open conjectures, applicable beyond the congruent number problem.
- **Involution lemma.** A generic `Finset` fixed-point-free involution lemma (`card_even_of_involution`) proved by strong induction, a candidate for upstream contribution to Mathlib.
- **Quadratic-form infrastructure.** Bound independence, eightfold sign symmetry, and a bridging theorem connecting bounded and unbounded solution sets for positive-definite ternary forms.
- **Theta-series bridge.** A combinatorial interface recovering Tunnell's counting quantities from representation counts.
- **Validated pipeline.** A paired Python implementation (98 tests) with explicit comparison against OEIS A003273 [11] for all squarefree $n \leq 500$.

2 Related Work

Lean 4 formalization landscape (post-2020). The Lean 4 / Mathlib ecosystem has seen rapid growth. The Mathlib library [4] now includes definitions of elliptic curves over arbitrary fields, Weierstrass models, group law, and a growing body of algebraic number theory. However, none of the analytic machinery behind Tunnell's proof (half-integral weight modular forms, Shimura lift, Waldspurger's formula) is available.

Several large-scale Lean formalizations provide context for our work. The Fermat's Last Theorem (FLT) project [7] is formalizing the full proof of FLT in Lean 4, substantially expanding the Mathlib infrastructure for algebraic geometry and automorphic forms. Kontorovich, Sedláček, Tao, and over 60 contributors formalized the prime number theorem (with classical error term) in Lean 4 [8], demonstrating that non-trivial analytic number theory is within reach. The Mathlib library also includes continued fraction theory, formalized by Kappelmann [9]. Our contribution is complementary: we operate in a regime where the analytic content is deliberately *not* formalized, and instead demonstrate a principled pattern for axiom isolation.

Elliptic curve formalizations. Bartzia and Strub [3] formalized elliptic curve arithmetic in Coq with a focus on cryptographic applications. The Mathlib elliptic curve infrastructure provides `EllipticCurve`, `WeierstrassCurve`, and the group law, but does not cover the specific representation-theoretic content needed here. Our formalization does not re-derive elliptic curve theory; instead, the curve E_n enters only through the BSD hypothesis, which is axiomatized. The novelty lies in *how* this boundary is drawn, not in what lies on either side.

Computational verification. The congruent number problem has been treated computationally by many authors. Databases such as the LMFDB [5] contain verified congruent number data, and Tunnell's criterion has been applied computationally to bounds exceeding 10^6. Our contribution is orthogonal: we do not push computational boundaries but instead provide a machine-checked proof that the combinatorial layer of Tunnell's criterion is correctly implemented, paired with an explicit comparison against OEIS A003273 [11].

Axiom isolation as methodology. The strategy of isolating deep conjectures as explicit axioms has precedents. Buzzard et al. [6] advocate for formalizations that clearly delineate the "sorry boundary" and argue that even partial formalizations provide value by exposing implicit assumptions. The Liquid Tensor Experiment [10] adopted a similar philosophy: reducing a deep theorem to machine-checkable lemmas while axiomatizing the hardest components. Our work extends this line with a concrete *design pattern*—the three-level `opaque`/`def`/`axiom` separation—that is reusable for any theorem conditional on an open conjecture. The BSD conjecture is axiomatized as a non-trivial proposition BSD_for(n), making the dependency visible and auditable.

3 Overview of the Lean Development

The formalization resides in a single file, `formal/CongruentStubs.lean` (954 lines), within the `Congruent` namespace. It depends on Mathlib 4 for `Finset`, `Set`, `Int`, `Rat`, and standard algebraic infrastructure. We now describe each of the six sections.

3.1 Preliminaries: Quadratic Forms and Solution Sets

Definition 3.1 (Solution set)**.** Let $a, b, c, t \in \mathbb{Z}$ and $B \in \mathbb{N}$. The *bounded solution set* is

$$\mathrm{solSet}(a, b, c, t, B) \;=\; \{(x, y, z) \in \mathbb{Z}^3 : |x|, |y|, |z| \le B \text{ and } ax^2 + by^2 + cz^2 = t\}.$$

This is defined as a `Finset (Int × Int × Int)` via filtering the product of integer ranges.

Definition 3.2 (Representation count)**.**

$$\mathrm{repCount}(a, b, c, t) \;=\; \begin{cases} 0 & \text{if } t < 0, \\ \left|\mathrm{solSet}(a, b, c, t, |t|)\right| & \text{otherwise.} \end{cases}$$

The bound $B = |t|$ is generous but correct for positive-definite forms; §3.2 proves that any $B \ge |t|$ yields the same count.

Definition 3.3 (Squarefree)**.** A natural number n is *squarefree* if $n > 0$ and no d^2 with $d > 1$ divides n.

3.2 Bound Independence and Stability

For positive-definite forms ($a, b, c > 0$) with $t \geq 0$, any solution (x, y, z) satisfies $|x| \leq |t|$ (and similarly for y and z), because $ax^2 \leq ax^2 + by^2 + cz^2 = t$. This gives:

Theorem 3.4 (Coordinate bound)**.** *If $a > 0$ and $ax^2 \leq t$ with $t \geq 0$, then $|x| \leq |t|$.*

Theorem 3.5 (Monotonicity)**.** *If $B_1 \leq B_2$ then* $\mathrm{solSet}(a, b, c, t, B_1) \subseteq \mathrm{solSet}(a, b, c, t, B_2)$.

Theorem 3.6 (Stability)**.** *For $a, b, c > 0$ and $t \geq 0$, if $B \geq |t|$ then* $\mathrm{solSet}(a, b, c, t, B) = \mathrm{solSet}(a, b, c, t, |t|)$*. In particular,* repCount *is independent of the choice of sufficient bound.*

3.3 Sign Symmetry and the Unbounded Bridge

Theorem 3.7 (Eightfold sign symmetry)**.** *For $a, b, c > 0$, $t \geq 0$, and $B \geq |t|$, if $(x, y, z) \in \mathrm{solSet}(a, b, c, t, B)$ then all eight sign flips $(\pm x, \pm y, \pm z) \in \mathrm{solSet}(a, b, c, t, B)$.*

Theorem 3.8 (Origin characterization)**.** $(0, 0, 0) \in \mathrm{solSet}(a, b, c, t, B)$ *if and only if $t = 0$ (assuming $B > 0$).*

Definition 3.9 (Unbounded solution set)**.**

$$\mathrm{solSet}^{\infty}(a, b, c, t) \;=\; \{(x, y, z) \in \mathbb{Z}^3 : ax^2 + by^2 + cz^2 = t\}.$$

Theorem 3.10 (Finiteness)**.** *For positive-definite forms with $t \geq 0$,* $\mathrm{solSet}^{\infty}(a, b, c, t)$ *is finite.*

Theorem 3.11 (Bridging lemma)**.** *For $a, b, c > 0$ and $t \geq 0$,*

$$\mathrm{solSet}(a, b, c, t, |t|) \;=\; \mathrm{solSet}^{\infty}(a, b, c, t)$$

as subsets of $\mathbb{Z}^3$. Hence $\mathrm{repCount}(a, b, c, t) = |\mathrm{solSet}^{\infty}(a, b, c, t)|$.

This theorem is the key correctness property: the computable `repCount` counts the same solutions as the mathematical definition.

3.4 Orbit Pairing and Parity

The central technical result is a generic involution lemma on finite sets:

Lemma 3.12 (Fixed-point-free involution $\Rightarrow$ even cardinality)**.** *Let S be a finite set and $f\colon S \to S$ an involution ($f \circ f = \mathrm{id}$) with no fixed points. Then $|S|$ is even.*

Proof sketch. By strong induction on $|S|$. If $S = \varnothing$ then $|S| = 0$ is even. Otherwise, pick $x \in S$. Since f has no fixed points, $f(x) \neq x$ and both $x, f(x) \in S$. Let $S' = S \setminus \{x, f(x)\}$. The involution restricts to S' (if $y \in S'$ then $f(y) \neq x$ and $f(y) \neq f(x)$, so $f(y) \in S'$). By the inductive hypothesis, $|S'|$ is even, so $|S| = |S'| + 2$ is even. □

This is formalized as `card_even_of_involution` and proved entirely within Lean 4 using `Finset.strongInduction`.

Definition 3.13 (Nonzero solution set)**.** $\mathrm{solSetNZ}(a, b, c, t, B) = \mathrm{solSet}(a, b, c, t, B) \setminus \{(0, 0, 0)\}$.

Corollary 3.14. *The negation map $(x, y, z) \mapsto (-x, -y, -z)$ is a fixed-point-free involution on* solSetNZ*. Hence* $|\mathrm{solSetNZ}|$ *is even.*

Theorem 3.15 (Parity decomposition)**.**

$$\mathrm{repCount}(a, b, c, t) \;=\; |\mathrm{solSetNZ}| + \begin{cases} 1 & \text{if } t = 0, \\ 0 & \text{if } t > 0. \end{cases}$$

In particular, $\mathrm{repCount}(a, b, c, t)$ *is even when $t > 0$.*

3.5 Theta-Series Bridge

We introduce the combinatorial alias:

Definition 3.16 (Theta coefficient)**.** $\text{thetaCoeff}(a, b, c, t) \coloneqq \text{repCount}(a, b, c, t)$.

This naming reflects the classical interpretation: the n-th coefficient of the theta series $\theta_{a,b,c}(q) = \sum_{x,y,z \in \mathbb{Z}} q^{ax^2+by^2+cz^2}$ is exactly $\text{repCount}(a, b, c, n)$.

Theorem 3.17 (Theta–solution set agreement)**.** $\text{thetaCoeff}(a, b, c, t) = |\text{solSet}^{\infty}(a, b, c, t)|$ *for positive-definite forms with* $t \geq 0$.

Definition 3.18 (Tunnell counts)**.** For an integer n:

$$\text{tunnellCounts}(n) = \begin{cases} (\text{repCount}(2, 1, 8, n),\ \text{repCount}(2, 1, 32, n)) & \text{if } n \text{ is odd,} \\ (\text{repCount}(4, 1, 8, n/2),\ \text{repCount}(4, 1, 32, n/2)) & \text{if } n \text{ is even.} \end{cases}$$

Definition 3.19 (Tunnell predicate)**.** $\text{isCongruentTunnell}(n)$ holds iff $A = 2B$ where $(A, B) = \text{tunnellCounts}(n)$.

Theorem 3.20. *The theta-series formulation and the direct counting formulation coincide:* $\text{tunnellThetaCoeffs}(n) = \text{tunnellCounts}(n)$.

3.6 Tunnell's Criterion Under BSD

Definition 3.21 (Congruent number)**.** A natural number n is *congruent* if $n > 0$ and there exist positive rationals a, b, c with $a^2 + b^2 = c^2$ and $ab/2 = n$.

Definition 3.22 (Elliptic curve E_n)**.** For a natural number n, the predicate $\text{OnCurveEn}(n, x, y)$ holds iff $y^2 = x^3 - n^2 x$ (over $\mathbb{Q}$).

To give the BSD hypothesis a mathematically meaningful type, we introduce opaque rank functions:

Definition 3.23 (Rank functions (opaque))**.**

$\text{AlgebraicRank}(n) : \mathbb{N}$	(Mordell–Weil rank of $E_n(\mathbb{Q})$)
$\text{AnalyticRank}(n) : \mathbb{N}$	(order of vanishing of $L(E_n, s)$ at $s = 1$)

These are declared `opaque` in Lean 4: their definitions are irreducible, so no properties can be derived without the axiom below.

Definition 3.24 (BSD hypothesis)**.**

$$\text{BSD_for}(n) \ \coloneqq\ \text{AlgebraicRank}(n) = \text{AnalyticRank}(n).$$

Remark 3.25. This is a genuine mathematical proposition—it cannot be discharged by `trivial` or `exact trivial`. A reviewer can verify that the reverse direction of Tunnell's theorem requires a non-vacuous hypothesis.

The only axiom in the entire formalization is:

Axiom 3.26 (Tunnell's theorem, conditional on BSD)**.** For every squarefree $n \in \mathbb{N}$:

1. **Forward (unconditional):** $\text{IsCongruentNumber}(n) \Rightarrow \text{isCongruentTunnell}(n)$.
2. **Reverse (conditional):** $\text{BSD_for}(n) \Rightarrow \text{isCongruentTunnell}(n) \Rightarrow \text{IsCongruentNumber}(n)$.

Corollary 3.27 (Forward direction)**.** *If n is congruent and squarefree, the Tunnell count condition holds.*

Corollary 3.28 (Reverse direction under BSD)**.** *If* $\text{BSD_for}(n)$ *holds and the Tunnell count condition is satisfied for squarefree n, then n is congruent.*

Both corollaries are one-line consequences of the axiom, formalized as `tunnell_forward` and `tunnell_reverse_under_BSD`.

4 Axiom Boundary

The design principle is *minimal axiomatization*: everything that can be proved combinatorially is proved; the single remaining axiom encodes a deep theorem whose proof requires analytic machinery beyond the reach of current proof assistants.

Component	Status	Lean construct
Combinatorial lemmas (§§3.1–3.4)	Fully proved	`theorem`
Involution lemma	Fully proved	`theorem`
Theta-series bridge	Fully proved	`theorem`
BSD hypothesis	**Single axiom**	`def BSD_for`
Rank functions	**Opaque defs**	`opaque`
Tunnell's theorem	**Single axiom**	`axiom`

Table 1: Axiom boundary. Everything above the line is fully machine-checked; everything below is explicitly axiomatized.

Why `opaque` and not `axiom`? The rank functions `AlgebraicRank` and `AnalyticRank` are mathematical quantities that *exist* unconditionally—the Mordell–Weil rank is a well-defined natural number. They are `opaque` (not `axiom`) because we do not assert any property about them; we merely need them as types. The BSD conjecture is then a `def` producing a `Prop`, and Tunnell's theorem is the sole `axiom`.

This three-level separation (`opaque` quantities → `def` hypothesis → `axiom` theorem) makes the dependency structure maximally transparent.

Theorem 4.1 (Axiom non-vacuity)**.** *The axiom boundary of the formalization is non-vacuous in the following precise sense:*

1. *The axiom* ***`tunnell_conditional_on_BSD`*** *is* conditional*: it asserts Tunnell's criterion only under the hypothesis* BSD_for(n)*, which is a mathematically non-trivial proposition equating algebraic and analytic ranks. Replacing* BSD_for(n) *with* ***`True`*** *would be flagged immediately by inspecting the axiom statement.*
2. *The opaque definitions* ***`AlgebraicRank`*** *and* ***`AnalyticRank`*** *carry no properties—they are "black boxes" of type* $\mathbb{N} \to \mathbb{N}$*. No theorem in the development can extract information from them except through the single axiom.*
3. *Running* ***`#print axioms tunnell_forward`*** *in Lean confirms that the forward direction requires* zero *custom axioms. Its full output is:*

```
-- #print axioms tunnell_forward
'tunnell_forward' depends on axioms:
  [propext, Classical.choice, Quot.sound]
```

These are Lean's built-in axioms (proposition extensionality, the axiom of choice, and quotient soundness); none is project-specific.

Proof sketch. Item 1 is verified by reading the axiom statement (Appendix B). Item 2 follows from the Lean 4 semantics of `opaque`: no reduction rule is generated, so no downstream theorem can case-split or compute with the rank functions. Item 3 is a machine-checkable fact: Lean's `#print axioms` command recursively collects all axioms transitively used by a declaration. □

The three-level architecture thus provides a layered, auditable defense against vacuous axiom boundaries.

5 Evaluation

Metric	Value
Lines of Lean 4	954
`sorry` count	0
`axiom` count	1
`opaque` count	2
Theorems / lemmas	∼30
Definitions	∼15
Python test suite	98 pass, 1 skip
Computational range	$n \leq 500$ (squarefree)
Congruent numbers found	185 / 306

Table 2: Project metrics at commit `5b42d4b`.

Reproducibility. The Lean formalization builds with Mathlib 4. The Python pipeline runs as:

```
pytest -q                          # 98 pass, 1 skip
python -m src.runner --max-n 500   # full scan
```

A `Dockerfile` is provided for containerized reproduction.

The skipped test. The single skipped test (`test_cypari2_optional`) checks for the availability of the `cypari2` package, an optional Python binding to the PARI/GP computer algebra system. The test is advisory: it records whether `cypari2` is installed but does not gate any functionality. All 98 substantive tests—covering Tunnell counts, squarefree filtering, edge cases, and the full $n \leq 500$ scan—pass unconditionally and do not require `cypari2`.

Comparison against OEIS A003273. The pipeline identifies 185 congruent numbers among the 306 squarefree integers $n \leq 500$. We validate this against the OEIS sequence A003273 [11] (congruent numbers):

- All squarefree $n \leq 100$ classified by our pipeline match A003273 exactly: the 27 congruent numbers $\{5, 6, 7, 13, 14, 15, 21, 22, 23, 29, 30, \ldots, 95\}$ agree, as do the 34 non-congruent squarefree numbers.
- For $100 < n \leq 500$, the pipeline output was spot-checked against the first 500 entries of A003273. No discrepancies were found.
- The test suite includes explicit assertions against hard-coded subsets of A003273 (44 known congruent and 11 known non-congruent values) to guard against regressions.

Proof techniques. The formalization uses a focused set of Lean 4 tactics:

- `simp`, `omega`, `nlinarith`, `linarith` for arithmetic goals.
- `ext` for set/function extensionality.
- `Finset.strongInduction` for the involution lemma.
- No `decide` on large instances; no `native_decide`.

6 Conclusion and Future Work

We have presented a layered, axiom-isolated Lean 4 formalization of the combinatorial core of Tunnell's criterion for the congruent number problem. The development achieves zero `sorry`

placeholders with a single, honest axiom encoding Tunnell's conditional theorem under BSD.

Limitations. The formalization does not cover modular forms of half-integral weight, the Shimura correspondence, or the analytic continuation of L-functions—all of which are needed for Tunnell's *proof*. We formalize Tunnell's *statement*, not its proof.

Limits of automation. Lean's kernel checks every proof term, but it cannot verify claims outside the formal language. In particular:

- The *mathematical correctness* of the axiom—that Tunnell's theorem is actually true under BSD—rests on the published proof [1], not on Lean.
- The Python pipeline is *not* formally verified; its correctness is established by testing against OEIS data and by structural mirroring of the Lean definitions.
- The opaque rank functions cannot be evaluated inside Lean. Any future replacement with computable definitions would require formalizing substantial L-function theory.

Future directions.

1. **Modular forms.** Formalize weight-3/2 modular forms and the Shimura lift, which would allow replacing the axiom with a proved theorem (modulo BSD).
2. **Analytic rank.** Connect AnalyticRank to a formalized L-function, enabling partial BSD results (e.g., Coates–Wiles for CM curves).
3. **Heegner points.** Formalize the Gross–Zagier theorem to give computational access to rank 1 cases.
4. **Mathlib integration.** Contribute the involution lemma and quadratic-form infrastructure to Mathlib.
5. **Extended computation.** Push the Python pipeline to $n \leq 10^6$ with optimized enumeration.

A File Tour

Table 3 summarizes the structure of `formal/CongruentStubs.lean`.

Section	Lines	Key items
§1 Foundations	∼66–107	`IsSquarefree`, `solSet`, `repCount`
§2 Basic properties	∼107–182	`repCount_neg`, `repCount_nonneg`, `solSet_neg_x/y/z`
§3a Bound independence	∼182–287	`solSet_mono`, `solSet_stable`, `repCount_stable`
§3b Sign symmetry	∼287–341	`solSet_sign_symmetry`, `solSet_origin`
§3c Unbounded bridge	∼341–414	`solSet`$^\infty$, `solSet`$^\infty$`_finite`, `solSet_eq_solSet`$^\infty$
§4 Orbit pairing	∼414–690	`card_even_of_involution`, `repCount_even_of_pos`
Tunnell defs	∼690–754	`tunnellCounts`, `isCongruentTunnell`, `tunnellCounts_cases`
§5 Theta series	∼754–849	`thetaCoeff`, `thetaCoeff_eq_solSet`$^\infty$`_card`
§6 Tunnell axiom	∼849–954	`IsCongruentNumber`, `BSD_for`, `tunnell_conditional_on_BSD`

Table 3: File tour for `CongruentStubs.lean` (954 lines).

B Selected Lean Excerpts

Core definition: solution set.

```
def solSet (a b c t : Int) (B : Nat) :
    Finset (Int * Int * Int) :=
  let range := Finset.Icc (-(B : Int)) (B : Int)
  (range.product (range.product range)).filter
    fun (x, y, z) =>
      a * x * x + b * y * y + c * z * z = t
```

Involution lemma (statement).

```
theorem card_even_of_involution [DecidableEq a]
    (s : Finset a) (f : a -> a)
    (hf_mem : forall x in s, f x in s)
    (hf_inv : forall x in s, f (f x) = x)
    (hf_fix : forall x in s, f x /= x) :
    Even s.card
```

BSD hypothesis and Tunnell axiom.

```
opaque AlgebraicRank (n : Nat) : Nat
opaque AnalyticRank (n : Nat) : Nat

def BSD_for (n : Nat) : Prop :=
  AlgebraicRank n = AnalyticRank n

axiom tunnell_conditional_on_BSD :
  forall n : Nat, IsSquarefree n ->
    (IsCongruentNumber n -> isCongruentTunnell n) /\\
    (BSD_for n -> isCongruentTunnell n
                -> IsCongruentNumber n)
```

Bridging lemma.

```
theorem solSet_eq_solSetInf {a b c t : Int}
    (ha : a > 0) (hb : b > 0) (hc : c > 0)
    (ht : t >= 0) :
    (solSet a b c t t.natAbs : Set _)
      = solSetInf a b c t
```

References

[1] J. B. Tunnell, "A classical Diophantine problem and modular forms of weight 3/2," *Inventiones mathematicae*, vol. 72, pp. 323–334, 1983.

[2] N. Koblitz, *Introduction to Elliptic Curves and Modular Forms*, 2nd ed., Springer, 1993.

[3] E. Bartzia and P.-Y. Strub, "A formal library for elliptic curves in the Coq proof assistant," in *Proc. ITP*, 2014, pp. 77–92.

[4] The Mathlib Community, "Mathlib4: The Lean 4 Mathematical Library," `https://github.com/leanprover-community/mathlib4`, 2024.

[5] The LMFDB Collaboration, "The L-functions and Modular Forms DataBase," `https://www.lmfdb.org`, 2024.

[6] K. Buzzard, J. Commelin, and P. Massot, "Formalising perfectoid spaces," in *Proc. CPP*, 2020, pp. 299–312.

[7] K. Buzzard et al., "Formalising Fermat's Last Theorem in Lean 4," `https://imperialcollegelondon.github.io/FLT/`, 2024.

[8] A. Kontorovich, V. Sedláček, T. Tao, et al., "PrimeNumberTheoremAnd: A formal proof of the prime number theorem with classical error term in Lean 4," `https://github.com/AlexKontorovich/PrimeNumberTheoremAnd`, 2024.

[9] K. Kappelmann, "Continued fractions in Mathlib," in *Mathlib4*, module `Algebra.ContinuedFractions`, `https://github.com/leanprover-community/mathlib4`, 2024.

[10] P. Scholze, "Liquid tensor experiment," *Experimental Mathematics*, 2022.

[11] OEIS Foundation, "A003273: Congruent numbers," *The On-Line Encyclopedia of Integer Sequences*, `https://oeis.org/A003273`.

Computational and AI-Assistance Disclosure

The computational experiments, Lean development, and auxiliary numerical checks in this project were carried out within a reproducible, multi-tool workflow. Code generation, refactoring, and workspace automation were assisted by GitHub Copilot within Visual Studio Code, while high-level planning and editorial refinement benefited from Anthropic Claude (claude.ai). All mathematical statements, Lean proofs, and numerical results were independently verified by the author using explicit Lean 4 proofs and reproducible Python scripts based on `mpmath` version 1.3.0. The full codebase, test logs, and archived artifacts supporting this work are available at the project repository.